Université / École d'Ingénieur

Département Informatique / Génie Logiciel

Projet de Fin d'Études

Rapport de Stage

Conception et Implémentation d'un Pipeline de Prétraitement Intelligent pour la Base de Connaissances STM32Cube

---

Réalisé par :
Abir GHARBI

Encadrant entreprise :
Nom du tuteur ST

Encadrant académique :
Nom du tuteur académique

Entreprise d'accueil : STMicroelectronics
Division : AI & Digital Tools

Année universitaire : 2025–2026

Remerciements

Je tiens à exprimer ma sincère gratitude envers toute l'équipe STMicroelectronics pour leur accueil, leur encadrement et leur soutien tout au long de ce stage. Je remercie particulièrement [nom du tuteur] pour sa disponibilité et ses conseils avisés.

Table des matières

Table des figures

Chapitre 1

Introduction Générale

1.1 Contexte

STMicroelectronics est un leader mondial dans la conception de semi-conducteurs. La famille STM32 représente plus de 1500 références de microcontrôleurs utilisés dans l'industrie, l'automobile, l'IoT et l'électronique grand public.

Les dépôts GitHub publics de STMicroelectronics (`github.com/STMicroelectronics`) contiennent :

— Des paquets logiciels complets (HAL, BSP, Middleware)

— Des milliers d'issues et pull requests (retours utilisateurs, corrections de bugs)

— De la documentation technique (Release Notes, exemples de code)

1.2 Problématique

Les ingénieurs et clients rencontrant des problèmes avec les microcontrôleurs STM32 n'ont pas de moyen efficace d'exploiter la richesse de ces données GitHub. La recherche manuelle dans les issues et la documentation est :

— **Lente** — des milliers d'issues par série MCU

— **Bruitée** — contenu non structuré, images, discussions hors-sujet

— **Fragmentée** — corrélations entre issues/PRs/commits non visibles

1.3 Objectifs du Projet

Concevoir et implémenter un pipeline automatisé de prétraitement qui :

1. Ingère les données brutes depuis l'API GitHub
2. Nettoie, enrichit et structure les documents

3. Génère des artefacts RAG-ready pour une base de connaissances
4. Alimente un chatbot IA spécialisé STM32 via l'API ST AI Bridge

1.4 Plan du Rapport

— **Chapitre 2** — État de l'art : RAG, NLP pour code, chatbots techniques
— **Chapitre 3** — Méthodologie et architecture du pipeline
— **Chapitre 4** — Implémentation détaillée
— **Chapitre 5** — Résultats et évaluation
— **Chapitre 6** — Conclusion et perspectives

Chapitre 2

État de l'Art

2.1 Retrieval-Augmented Generation (RAG)

Le paradigme RAG combine la recherche d'information avec la génération de texte par des LLMs [?]. Au lieu de se fier uniquement aux connaissances paramétrisées du modèle, RAG :

1. **Récupère** des documents pertinents depuis une base de connaissances
2. **Augmente** le contexte du prompt avec ces documents
3. **Génère** une réponse fondée sur des preuves factuelles

2.1.1 Avantages pour le domaine technique

— Réduction des hallucinations grâce aux sources vérifiables

— Mise à jour continue de la KB sans ré-entraînement

— Traçabilité des réponses vers les issues/PRs source

2.2 Prétraitement pour Knowledge Bases

La qualité du RAG dépend directement de la qualité des documents indexés :

2.2.1 Chunking Strategies

— **Fixed-size** : simple mais coupe le contexte

— **Paragraph-based** : respecte les frontières sémantiques

— **Section-based** : adapté aux documents structurés

— **Overlap** : assure la continuité contextuelle entre chunks

2.2.2 Enrichissement Métadonnées

L'ajout de métadonnées structurées (composant, couche, sévérité) permet un filtrage précis lors de la recherche.

2.3 Traitement du Langage Naturel pour Code Source

— TF-IDF pour la similarité inter-issues

— Heuristiques regex pour la détection de composants matériels

— Classification supervisée vs. règles pour le type de problème

2.4 Solutions Existantes

Solution	**Forces**	**Limites**
GitHub Copilot	Code completion	Pas de KB custom
Stack Overflow Search	Communauté large	Pas spécifique ST
RAG générique	Flexible	Pas de prétraitement domain-aware
Notre approche	Domain-specific + enrichi	Requiert pipeline custom

Chapitre 3

Méthodologie et Architecture

3.1 Approche Méthodologique

Le projet suit une approche itérative :

1. **Phase 1** : Pipeline H7 (série pilote, 12 repos)
2. **Phase 2** : Extension multi-séries (F4, H5, U5, WL)
3. **Phase 3** : Enrichissement IA (Alfred Vision) et évaluation

3.2 Architecture Globale

Le système s'articule autour de 3 couches :

1. **Couche Data** : ingestion et stockage structuré (JSON)
2. **Couche Processing** : pipeline Python en 7 étapes
3. **Couche Delivery** : export et upload vers la KB cloud

3.3 Choix Techniques

Choix	**Justification**
Python 3.14	Écosystème riche (requests, scikit-learn), déploiement simple
JSON comme format pivot	Natif pour APIs REST, lisible, versionnable
TF-IDF (scikit-learn)	Léger, rapide, suffisant pour la similarité textuelle
Heuristiques vs ML	Déterministe, explicable, pas de données d'entraînement requises

Architecture modulaire	Chaque étape est rejouable indépendamment

3.4 Diagramme de Flux de Données

(Voir diagrammes draw.io en annexe : 01_pipeline_overview.drawio)

```
GitHub API → 01_raw → 02_clean → 03_enriched → 04_similarity
         → 05_chunked → 06_evaluated → 07_delivery → KB #793
```

Chapitre 4

Implémentation

4.1 Ingestion (Stage 1)

4.1.1 Objectif

Récupérer exhaustivement les données des repositories GitHub configurés.

4.1.2 Mécanisme

— Pagination automatique (100 items/page)

— Gestion du rate-limiting GitHub (5000 req/h avec token)

— Retry exponentiel en cas d'erreur réseau

— Fetch parallélisable par repository

4.1.3 Données récupérées

Pour chaque repo : issues (avec commentaires), PRs, commits, liens croisés, et fichiers source.

4.2 Nettoyage (Stage 2)

— Suppression des issues bot/spam

— Normalisation des labels GitHub

— Extraction du texte utile (suppression des templates vides)

— Déduplication par identifiant

4.3 Enrichissement V2 (Stage 3)

4.3.1 Heuristiques de Classification

Le module `stm32cube_preprocessing_heuristics.py` applique des règles basées sur :

— **Regex patterns** : détection de séries MCU, composants, boards

— **Analyse des labels** : mapping label→layer (bug→bug_report, etc.)

— **Scoring pondéré** : longueur du body, présence de code, de logs d'erreur

4.3.2 Exemple de Classification

```
{
  "issue_number": 42,
  "title": "HAL_SPI_TransmitReceive DMA fails on STM32F446",
  "layer": "HAL",
  "component": "SPI",
  "board": "NUCLEO-F446RE",
  "issue_kind": "bug_report",
  "severity": "high",
  "evidence_strength": "high"
}
```

4.4 Similarité (Stage 4)

4.4.1 Algorithme

1. Construction de la matrice TF-IDF sur le corpus (titre + body + commentaires)
2. Calcul de la similarité cosinus paire-à-paire
3. Sélection des top-5 voisins (seuil > 0.3)

Formule de similarité cosinus :

$$\text{sim}(A, B) = \frac{A \cdot B}{\|A\| \times \|B\|}$$

4.5 Chunking V2 (Stage 5)

Stratégie de découpage :

— Issues : chunks par paragraphe, max 512 tokens, overlap 50 tokens

— Files : chunks par section (README headers, fonctions C)

4.6 Delivery (Stage 7)

4.6.1 Resolver Cases

Générés à partir d'issues fermées avec PRs liées :

1. Identification de la chaîne Issue → PR → Commits
2. Évaluation du `resolution_status` (merged_fix, candidate_fix, reference_only)
3. Construction du texte structuré : Problème → Signal → Évaluation → Preuves

4.6.2 Diagnostic Cards

Cartes de diagnostic structurées pour les problèmes récurrents :

1. Sélection des issues à haute valeur diagnostique
2. Génération : Symptôme → Cause Racine → Résolution → Validation
3. Alias de requête pour améliorer la récupération

Chapitre 5

Résultats et Évaluation

5.1 Couverture des Données

Série	Repos	Issues	Files	Resolver Cases
STM32F4	16	∼800	∼5400	∼150
STM32H5	6	∼200	∼1200	∼40
STM32U5	8	∼350	∼2000	∼70
STM32WL	5	∼150	∼800	∼30
STM32H7	12	∼500	∼3000	∼100
Total	**47**	**∼2000**	**∼12400**	**∼390**

5.2 Qualité de l'Enrichissement

Taux d'enrichissement réussi (champ non "unknown") :

— **Layer** : ∼85% des issues

— **Component** : ∼70% des issues

— **Board** : ∼45% (dépend de la mention explicite)

— **Severity** : 100% (toujours calculé)

5.3 Performance de Recherche

Tests TF-IDF sur requêtes techniques :

— Précision@5 : ∼0.72 (issues), ∼0.78 (files)

— La recherche hybride (semantic + full-text) sur la KB améliore significativement vs. TF-IDF seul

5.4 Test Suite Qualité (50 Questions)

Évaluation sur 50 questions réalistes couvrant les 4 séries (F4, H5, U5, WL) :

— Questions formulées en langage naturel “client”

— Chaque question inclut contexte board + symptôme technique

— Réponse attendue avec ancrage technique vérifiable

Chapitre 6

Conclusion et Perspectives

6.1 Bilan

Ce projet a permis de concevoir et déployer un pipeline de prétraitement complet qui :

— Transforme des données GitHub brutes en connaissances structurées

— Alimente un chatbot IA spécialisé STM32 en production

— Couvre 5 séries MCU (47 repositories, ∼14400 documents)

— Intègre l'enrichissement IA par vision (Alfred)

6.2 Perspectives

1. **Extension à d'autres séries** : STM32L4, STM32G4, STM32MP
2. **Enrichissement sémantique** : embeddings pré-calculés pour la similarité
3. **CI/CD automatisé** : Jenkins pipeline pour mise à jour continue
4. **Évaluation automatique** : scoring RAG avec métriques (Faithfulness, Relevance)
5. **Multi-langue** : support des issues en chinois/japonais

Références

Bibliographie

[1] Lewis, P. et al. (2020). *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*. NeurIPS 2020.

[2] OpenAI (2023). *GPT-4 Technical Report*. arXiv :2303.08774.

[3] Salton, G., Buckley, C. (1988). *Term-weighting approaches in automatic text retrieval*. Information Processing & Management.

[4] STMicroelectronics (2024). *STM32Cube MCU & MPU Packages*. https://github.com/STMicroelectronics

Annexes

A. Liste des Diagrammes

- docs/diagrams/presentation/01_pipeline_overview.drawio
- docs/diagrams/presentation/02_delivery_kb_upload.drawio
- docs/diagrams/presentation/03_data_flow_source_to_answer.drawio
- docs/diagrams/technical/01_ingestion_detail.drawio
- docs/diagrams/technical/02_enrichment_similarity_detail.drawio
- docs/diagrams/technical/03_delivery_resolver_diagnostic.drawio

B. Commandes d'Exécution

```
# Activer l'environnement
.venv\Scripts\activate

# Pipeline complet avec delivery
python -m pipeline.run_full_workflow --export-st-ready

# Validation des schemas
python -m pipeline.evaluation.validate_schemas

# Upload vers KB
python Add_Data_Source_Files.py --operation add \
  --ds-id 28099 --json-split-size 1000 \
  --file "datasets/07_delivery/st_ready/.../issues.json"
```

C. Structure JSON — Resolver Case (Extrait)

```
{
  "resolver_cases": [
```

```
    {
      "issue_number": 42,
      "repo": "stm32cubef4",
      "label": "SPI DMA TransmitReceive fails...",
      "resolution_status": "merged_fix",
      "link_confidence": "high",
      "primary_pr_number": 87,
      "primary_changed_file": "stm32f4xx_hal_spi.c",
      "resolver_card_text": "Problem: ...",
      "impacted_modules": ["SPI", "DMA"],
      "impacted_boards": ["NUCLEO-F446RE"]
    }
  ]
}
```